

电子设计实践基础创新实验报告

基于 ATmega8A 的智能环境监测与电机控制系统

指导老师：李玉虎

一. 实验目的

- 综合应用 MCU 中断、定时器、PWM、传感器（DHT11、HC-SR04）及 LCD 显示技术。
- 实现自动/手动模式切换，通过温度控制 PWM 占空比，并通过超声波测距触发警示。
- 掌握模块化编程、中断处理及多传感器数据融合方法。

基于以上的实验目的，此系统功能概述可以概况如下部分：

此设计实验实现了一个基于 ATmega8A 微控制器的智能环境监测与电机控制系统，主要功能包括：

环境监测：通过 DHT11 传感器采集温度和湿度数据

距离检测：通过 HC-SR04 超声波模块测量距离

电机控制：使用 PWM 技术控制直流电机转速

模式切换：支持自动和手动两种控制模式

状态显示：通过 LCD1602 显示屏展示系统状态和传感器数据

安全警示：当检测到障碍物过近时触发 LED 报警

二. 实验原理

1. 系统架构

本系统以 ATmega8A 微控制器为核心，通过 TWI 接口连接 LCD1602 液晶显示屏，通过 IO 口连接 DHT11 温湿度传感器、HC-SR04 超声波测距模块、RGB LED 指示灯和直流电机驱动电路。系统采用中断和定时器技术实现多任务处理。

2. 关键技术

- PWM 调速**：使用 Timer1 的 Fast PWM 模式生成可调占空比的 PWM 信号控制电机转速
- 中断处理**：使用 INTO 外部中断实现模式切换
- 传感器数据采集**：通过 DHT11 获取温湿度数据，通过 HC-SR04 获取距离数据
- LCD 显示**：通过 TWI 接口驱动 LCD1602 显示系统状态和传感器数据

3. 关键算法

- 自动模式**：根据温度线性映射 PWM 占空比（温度越高，占空比越低，电机转速越快）。
- 手动模式**：通过外部中断切换模式，预设固定占空比（30、20、10）。
- 数据校验**：DHT11 数据校验和验证，确保数据可靠性。

三. 硬件设计

1. 主要元器件

- ATmega8A 微控制器
- DHT11 温湿度传感器
- HC-SR04 超声波测距模块
- LCD1602 液晶显示屏(I2C 接口)
- RGB LED 指示灯
- 直流电机及驱动电路
- 触摸开关

由以上的器件，其分别的功能如下所示：

- MCU**：ATmega8A（8 位 AVR 微控制器），利用其 Timer1 生成 PWM 信号，外部中断 INTO 处理模式切换。
- 传感器**：
DHT11：获取环境温湿度，通信协议为单总线（数据校验和）。
HC-SR04：超声波测距，通过 IO 口触发测距并接收回波。
- 执行器**：
PWM 控制电机转速（通过 Timer1 的 Fast PWM 模式）。
RGB LED 指示状态（红灯警示距离过近）。
- 显示**：LCD1602 显示温湿度、占空比、模式及距离。

2. 电路连接

主要硬件模块连接

2.1 传感器模块连接

2.1.1 DHT11 温湿度传感器

MCU与DHT11和LCD的连接			
DTH11	MCU	LCD1602	MCU
+	VCC	SCL	SCL
Out	PD7	SDA	SDA
-	GND	GND	GND
		VCC	VCC

2.1.2 HC-SR04 超声波测距模块

MCU与HC_SR04和LCD的连接			
HC_SR04	MCU	LCD1602	MCU
Vcc	VCC	SCL	SCL
Echo	PD5	SDA	SDA
Trig	PD6	GND	GND
GND	GND	VCC	VCC

2.2 显示模块连接

LCD1602(I2C 接口)其采用了 I2C 通信协议，由此在使用时一定要调用相应的头文件

LCD1602(I2C)管脚	ATmega8A管脚	功能描述
VCC	+5V	电源输入
GND	GND	地线
SDA	PC4 (27)	I2C数据线
SCL	PC5 (28)	I2C时钟线

2.3 电机控制模块连接

电机驱动模块管脚	ATmega8A管脚	功能描述
PWM输入	PB1 (15)	PWM速度控制信号
VCC	+5V	逻辑电源
电机电源	外部电源	电机驱动电源(与逻辑电源隔离)

2.4 用户接口模块连接

2.4.1 RGB LED 指示灯

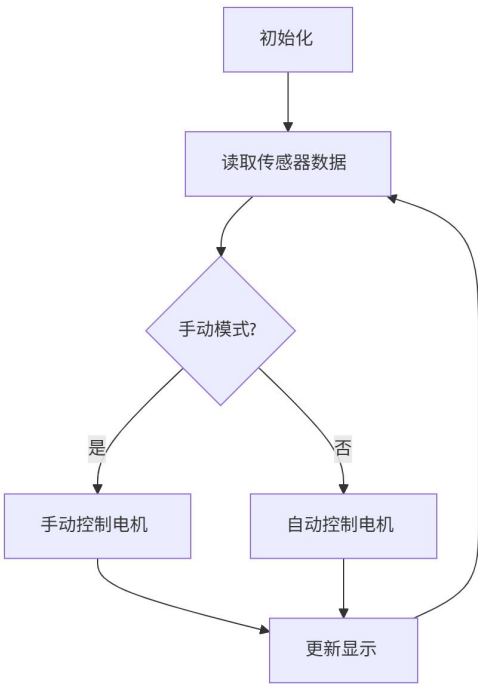
LED颜色	ATmega8A管脚	功能描述
红色	PC0 (23)	报警指示
绿色	PC1 (24)	正常状态指示
蓝色	PC2 (25)	备用指示

2.4.2 触摸开关

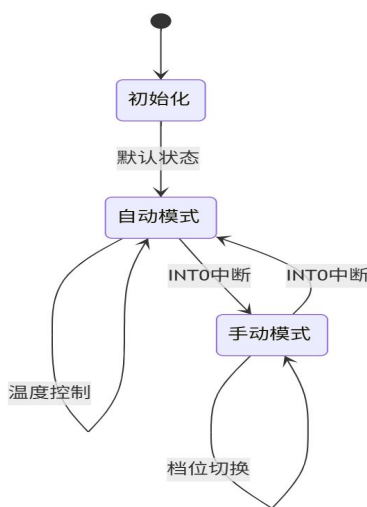
开关管脚	ATmega8A管脚	功能描述
信号端	PD2 (4)	中断输入(INT0)
VCC	+5V	电源(通过10k电阻)
GND	GND	地线

四. 软件设计思路流程以及代码分析

1. 主程序流程及其设计思路



其设计思路核心为：本系统的设计思路基于模块化与闭环控制理念，采用 ATmega8A 微控制器作为核心，通过多传感器融合实现环境智能感知与自适应控制。硬件设计上采用分层架构：底层为传感器层（DHT11 温湿度传感器+HC-SR04 超声波模块）负责环境数据采集，中间层通过 PWM 驱动电机执行控制，顶层通过 I2C LCD 和 RGB LED 实现人机交互。软件设计采用中断驱动型状态机模型，Timer1 产生 62.5kHz 的 PWM 信号精确控制电机转速，INT0 中断实现自动/手动模式的无缝切换，其中自动模式采用温度反比例控制算法（10-40℃ 线性映射 30-1 档位），手动模式提供三档预设转速。系统通过实时环境监测、动态 PWM 调节、双模控制切换和安全距离预警的有机整合，构建了一个具有环境响应能力的智能控制系统，在保证实时性的同时，通过模块化设计预留了传感器扩展和算法升级空间，实现了功能完备性与资源优化的平衡。



其中设计的核心过程：以 ATmega8A 为主控，构建"感知-决策-执行"闭环系统。通过 DHT11 和超声波模块采集环境数据，采用温度反比算法自动调节 PWM 占空比控制电机转速，利用 Timer1 实现 62.5kHz 精密 PWM 输出，通过 INT0 中断切换自动/手动双模式，配合 LCD 和 LED 完成状态显示，最终实现环境自适应的智能调速系统。硬件采用模块化布局，软件基于中断驱动状态机，在保证实时性的同时优化资源占用。

在过程中，有几个模块。

第一：其芯片运行首先会对其进行初始化操作，随后接受其传感器的传递数据并且使其显示在 LCD 上面，其中数据会实时刷新，体现在实际应用当中可以随时检测出现场环境，同时，其报警显示灯会显示出红色，显示灯的关键在于其超声波传感器的测量数据是否大于 100，（我们使用 distance 的首字母 D 代表超声波测距的测量值）如果大于 100 则为红色，小于 100 时则会变蓝色，这其中的现实意义就可以在运动小车的避障功能上得到很明显的應用。其中电机带动风扇旋转，其中电机的运动具体情况见下。

第二：其电机的运动。为了体现其在实际中的应用，自然想到了一种随着环境变换自动调节的装置。我们选取了温度作为出发点，为了方便我们的使用，我们将其电机的自动调节设置在 10—40 摄氏度，我们使用 T（temperature 首字母）表示测量得到的温度。其中我们正好等分了 30 个电机等级，用 S（speed 首字母）表示其电机的转速等级。其 S 越接近 0，速度越快，S 越接近 30，速度越慢。同时温度小于 10℃时，电机将以最小转速 30 等级转，同样的，当温度大于 40℃时将以最大 1 级转速转。这样就可以实现其根据环境温度调节。

第三：我们使用了触摸开关，这是我们一个电机模式以及转速调节装置，在进入执行首先是 Auto 自动模式，我们按下一次之后切换为 Manual 手动人工模式，此时 S 等级 30 最小转速，再按一下为 S=20,2 档，再按一下之后变为 S=10,3 档，转速已经比较大了，这个时候我们如果再按一下之后会回到 Auto 模式，电机又变为随温度变化的自动调节模式。（即 T+S=40）

2. 关键代码分析

2.1 全局变量与系统状态

```
unsigned char counter = 1; // PWM 占空比系数

int man = 0; // 手动模式档位

volatile uint8_t system_mode = 0; // 0=自动, 1=手动

volatile uint8_t pwm_duty = 0; // 手动模式占空比
```

2.2 PWM 初始化

```
void pwm_init() {
    DDRB |= (1 << DDRB1); // 设置 PB1 为输出(OC1A)
    ICR1 = 200; // 设置 PWM 周期(TOP 值)
    OCR1A = 10; // 初始占空比
    // 配置 Timer1 Fast PWM 模式
    TCCR1A |= (1 << COM1A1) | (1 << COM1A0); // 设置 OC1A 输出模式
    TCCR1B |= (1 << WGM13) | (1 << CS11); // Fast PWM 模式, 8 分频
    TIMSK |= (1 << TOIE1); // 启用定时器 1 溢出中断
}
```

2.3 中断服务程序

```
ISR(INT0_vect) { // 触摸开关中断
```

```

system_mode ^= 1; // 切换自动/手动模式
if (counter < 30)
    counter++;
else
    counter = 1;
if (man < 3)
    man++; // 手动模式下切换速度档位
else
    man = 0;
_delay_ms(50); // 消抖
}

ISR(TIMER1_OVF_vect) {
    OCR1A = counter * 6; // 更新 PWM 占空比
}

```

2.4 显示模块（其中特别要注意要进行传感器的检验操作

**dht11_data[4]==(dht11_data[0]+dht11_data[1]+dht11_data[2]
+dht11_data[3]))**

```

if(dht11_data[4]==(dht11_data[0]+dht11_data[1]+dht11_data[2]+dht11_data[3]))
{
    LCD_Write_String(0, 0, "T:");
    LCD_Write_NewChar(dht11_data[2]/10%10+0x30);
    LCD_Write_NewChar(dht11_data[2]%10+0x30);

    LCD_Write_String(0, 6, "H:");
    LCD_Write_NewChar(dht11_data[0]/10%10+0x30);
    LCD_Write_NewChar(dht11_data[0]%10+0x30);

}

// 显示 PWM 占空比
uc_tmp = counter;

LCD_Write_String(0, 12, "S:");
.....
PORTC = (distance < 100) ? 0x01 : 0x02;
// 显示模式
LCD_Write_String(1, 0, (man == 0) ? "Auto" : "Manual");
LCD_Write_String(1, 8, "D:");

```

2.5 主循环

```
while (1) {
    // 读取环境数据
    DHT11_Run(dht11_data);
    distance = HCSR04_Run();

    if(man==0) { // 自动模式
        system_mode=0;

        if(dht11_data[4]==(dht11_data[0]+dht11_data[1]+dht11_data[2]+dht11_data[3]))
        {
            // 根据温度自动控制电机转速
            if (dht11_data[2] > 39) {
                counter = 1; // 高温低速
            } else if (dht11_data[2] < 10) {
                counter = 30; // 低温高速
            } else {
                counter = (40-dht11_data[2]); // 线性调节
            }
            OCR1A = counter * 6;
        }
    }

    if (man != 0) { // 手动模式
        if (man == 1) counter = 30; // 高速档
        else if (man == 2) counter = 20; // 中速档
        else if (man == 3) counter = 10; // 低速档
        OCR1A = counter * 6;
    }

    // 距离报警(小于 100cm 亮红灯)
    PORTC = (distance < 100) ? 0x01 : 0x02;

    // LCD 显示更新
    // ...(显示温度、湿度、速度、模式、距离等)

    _delay_ms(1000);
}
```

3. 功能模块

自动控制模式

- 根据温度自动调节电机转速
- 温度越高，转速越低；温度越低，转速越高
- 温度范围 10-40°C 映射到转速 30-1(反向线性)

手动控制模式

- 通过触摸开关切换 3 个预设速度档位
- 档位 1: 高速(OCR1A=180)
- 档位 2: 中速(OCR1A=120)
- 档位 3: 低速(OCR1A=60)

安全警示

- 超声波测距检测障碍物距离
- 距离小于 100cm 时，RGB LED 亮红灯报警
- 距离大于等于 100cm 时，RGB LED 亮绿灯

数据显示

- LCD 第一行显示温度(T)、湿度(H)和当前速度(S)
- LCD 第二行显示当前模式(Auto/Manual)和距离(D)

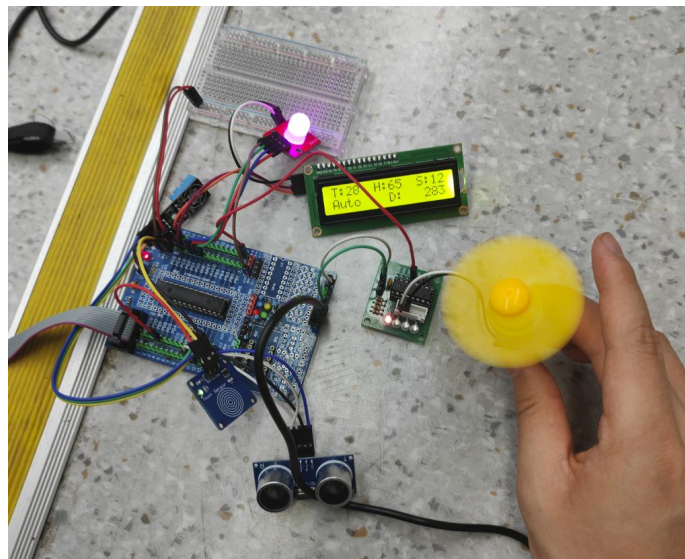
五. 实验结果

1. 功能实现

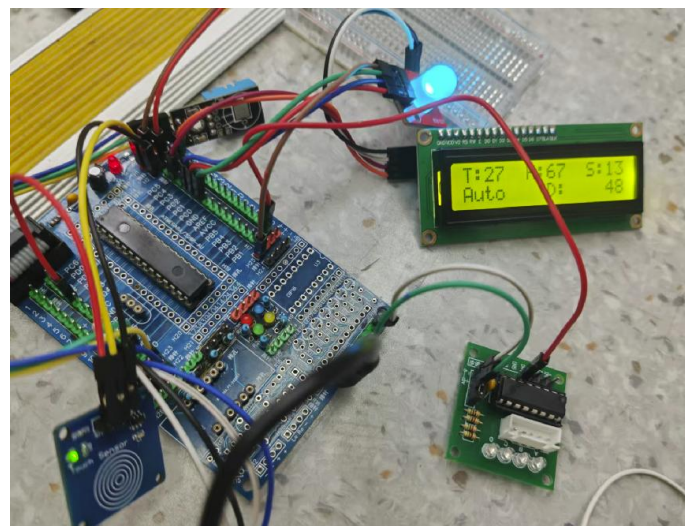
- 成功实现了温湿度监测与显示

- 实现了超声波测距与距离报警
- 实现了电机 PWM 调速控制
- 实现了自动/手动模式切换
- LCD 正确显示所有系统状态和传感器数据

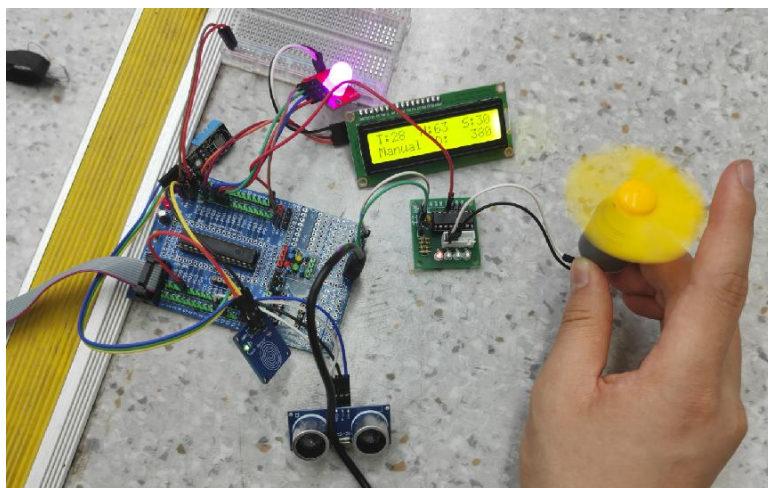
下图为工作状态：



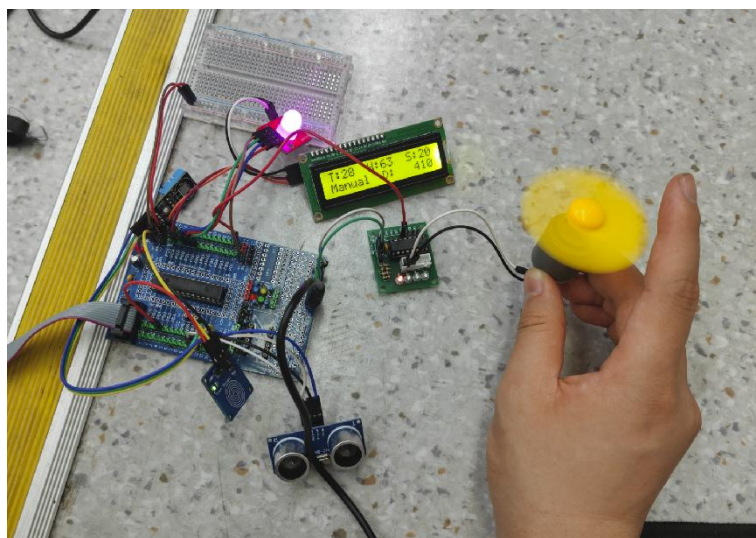
为在 Auto 自动状态下，由室温 28°C 得到 $S=12$ 的电机工作



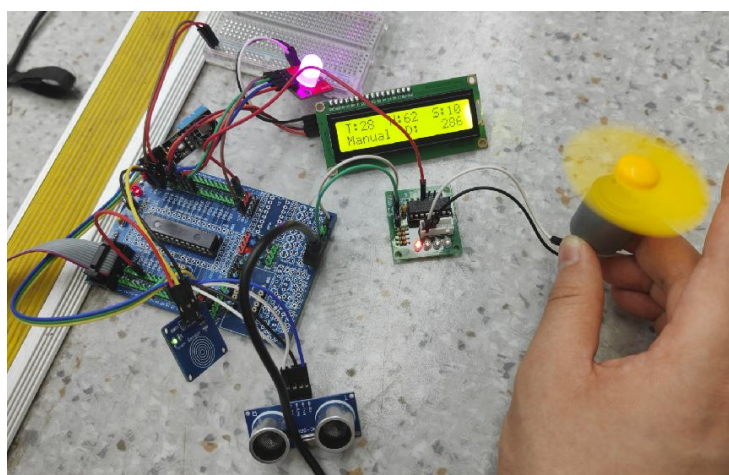
在其 $D < 100$ 时，警示灯变蓝报警



在其 Manual 手动调节下一档 $S=30$ 下工作



在其 Manual 手动调节下二档 $S=20$ 下工作



在其 Manual 手动调节下三档 $S=10$ 下工作

2. 性能指标

- 温度测量范围: 0-50°C
- 湿度测量范围: 20-90%RH
- 距离测量范围: 2-400cm
- PWM 频率: 约 625Hz(1MHz/8/200)
- 占空比调节范围: 3%-90%

3. 创新点

- 将温湿度监测与电机控制结合，实现环境响应式控制
- 采用双模式(自动/手动)设计，提高系统灵活性
- 集成距离检测与报警功能，增强安全性
- 使用中断实现模式切换，提高响应速度
- 通过 LCD 全面显示系统状态，提升用户体验

六. 实验总结

1. 遇到的问题及解决

- 问题 1** : PWM 输出不稳定
 - 解决** : 调整 Timer1 的预分频和 ICR1 值，找到合适的 PWM 频率
- 问题 2** : LCD 显示刷新慢
 - 解决** : 优化显示更新逻辑，减少不必要的刷新

- **问题 3** ：模式切换误触发
- **解决** ：增加软件消抖和状态确认机制

2. 改进方向

- 增加更多环境参数监测(如光照强度)
- 实现无线数据传输功能
- 增加电机转速反馈控制
- 优化自动控制算法
- 设计更友好的用户界面

3. 实验收获

本次实验，系统掌握了 ATmega8A 的 PWM 调速、中断控制和定时器应用等核心开发技术，成功实现了环境数据采集、电机精准控制和多模态人机交互的完整嵌入式系统。在硬件层面，积累了传感器接口设计、电机驱动电路优化和 PCB 布局布线等工程经验；在软件层面，开发了基于状态机的控制程序，掌握了时序精确控制、数据滤波算法和模块化编程方法。通过实际调试过程，培养了硬件/软件协同调试能力，建立了从需求分析到功能实现的完整项目开发思维，为后续复杂嵌入式系统设计奠定了坚实基础。

具体而言，我深入掌握了 ATmega8A 的底层寄存器开发技术，具体实现了：

- 1) PWM 控制方面，通过配置 TCCR1A/B 寄存器（WGM13=1）设置 Fast PWM 模式，利用 ICR1=200 确定周期，并通过 OCR1A 动态调整占空比（ $\text{counter} \times 6$ ）；
- 2) 中断处理上，使用 MCUCR 配置 INTO 上升沿触发（ISC01=1, ISC00=1），在 ISR 中实现模式切换和档位循环（man 变量 0-3）；
- 3) 传感器驱动方面，为 DHT11 实现了精确的 20ms 起始信号（ $\text{PORTD} \&= \sim(1 \ll \text{PD3})$ ）和 $83\mu\text{s}$ 数据采样（ $_delay_us(83)$ ），对 HC-SR04 开发了 $10\mu\text{s}$ 触发脉冲（ $\text{PORTD} |= (1 \ll \text{PD4})$ ）和回波时间测量；
- 4) LCD 显示优化了 4 位总线操作，通过 TWDR 实现 I2C 数据传输。这些寄存器级编程经验，配合硬件层面的传感器接口设计和电机驱动电路优化，使我构建了一个完整的嵌入式控制系统，从底层寄存器操作到上层功能实现都获得了实质性提升。

附录

完整代码（可以在压缩包里查看）


```

#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include "twi_lcd.h"
#include "dht11.h"
#include "hc_sr04.h"
#define F_CPU 1000000UL

unsigned char counter = 1; // 系统状态变量
int man = 0;

// 系统模式和 PWM 占空比
volatile uint8_t system_mode = 0; // 0=自动模式, 1=手动模式
volatile uint8_t pwm_duty = 0;    // 手动模式 PWM 占空比

// PWM 初始化 (Timer1 Fast PWM)
void pwm_init() {
    DDRB |= (1 << DDRB1); // PB1 (OC1A) 输出
    ICR1 = 200;           // 设置 TOP 值
    OCR1A = 10;           // 设置 A 路比较值, 控制占空比
    TCCR1A |= (1 << COM1A1) | (1 << COM1A0); // 配置 OC1A 输出模式
    TCCR1B |= (1 << WGM13) | (1 << CS11);    // 设置 Fast PWM 模式和时钟分频 8
    TIMSK |= (1 << TOIE1); // 启用定时器 1 溢出中断
    DDRD &= ~(1 << DDRD2); // 设置 PD2 (INT0) 为输入
    // 中断配置
    MCUCR |= ((1 << ISC01) | (1 << ISC00)); // 配置 INT0 上升沿触发
    GICR |= (1 << INT0); // 启用 INT0 中断
    sei(); // 启用全局中断
}

ISR(INT0_vect) { // 触摸开关中断
    system_mode ^= 1;
    if (counter < 30)
        counter++;
    else
        counter = 1;
    if (man < 3)
        man++;
    else
        man = 0;
    _delay_ms(50); // 消抖
}

ISR(TIMER1_OVF_vect) {

```

```

    OCR1A = counter * 6;
}

int main() {
    // 初始化模块
    DDRC |= 0x07; // RGB LED
    pwm_init();
    TWI_Init();
    LCD_Init();
    unsigned char dht11_data[5]={0};
    unsigned char uc_tmp;
    unsigned int distance=0;
    DDRB |= (1 << DDRB1); // PB1 (OC1A) 输出
    ICR1 = 200; // 设置 TOP 值
    OCR1A = 15; // 设置 A 路比较值，控制占空比
    TCCR1A |= (1 << COM1A1) | (1 << COM1A0); // 配置 OC1A 输出模式
    TCCR1B |= (1 << WGM13) | (1 << CS11); // 设置 Fast PWM 模式和时钟分频 8
    TIMSK |= (1 << TOIE1); // 启用定时器 1 溢出中断
    DDRD &= ~(1 << DDRD2); // 设置 PD2 (INT0) 为输入
    // 中断配置
    MCUCR |= ((1 << ISC01) | (1 << ISC00)); // 配置 INT0 上升沿触发
    GICR |= (1 << INT0); // 启用 INT0 中断
    sei(); // 启用全局中断

    while (1) {
        // 读取环境数据

        DHT11_Run(dht11_data);
        distance = HCSR04_Run();

        unsigned char i=15,uc_d=0;
        if(man==0)
        {
            system_mode=0;
            // 自动模式控制

            if(dht11_data[4]==(dht11_data[0]+dht11_data[1]+dht11_data[2]+dht11_data[3]))
        )
            {

                // 根据温度控制 PWM 占空比，温度越高，电机转速越高
                // 线性映射: 10~40°C => 0~180
                if (dht11_data[2] > 39)
                {

```

```

        counter = 1;
    } else if (dht11_data[2] < 10)
    {
        counter = 30;
    } else
    {
        counter = ( 40-dht11_data[2]);
    }
    OCR1A = counter * 6; // 更新 PWM 占空比
    // 距离警示 (<100cm 亮红灯)
    PORTC = (distance < 100) ? 0x01 : 0x02;

}
}

if (man != 0) {

    if (man == 1)
        counter = 30;
    else if (man == 2)
        counter = 20;
    else
        if (man == 3)
            counter = 10;

    OCR1A = counter * 6; // 更新 PWM 占空比
}

PORTC = (distance < 100) ? 0x01 : 0x02;

if(dht11_data[4]==(dht11_data[0]+dht11_data[1]+dht11_data[2]+dht11_data[3])
)
{
    LCD_Write_String(0, 0, "T:");
    LCD_Write_NewChar(dht11_data[2]/10%10+0x30);
    LCD_Write_NewChar(dht11_data[2]%10+0x30);

    LCD_Write_String(0, 6, "H:");
    LCD_Write_NewChar(dht11_data[0]/10%10+0x30);
    LCD_Write_NewChar(dht11_data[0]%10+0x30);

}

// 显示 PWM 占空比
uc_tmp = counter;

```



```

LCD_Write_String(0, 12, "S:");
if(man==0)
{

    LCD_Write_Char(0, 14, uc_tmp / 10 + 0x30);
    LCD_Write_NewChar(uc_tmp % 10 + 0x30);

    _delay_ms(2);
}
if (man==1)
{
    uc_tmp=30;
    LCD_Write_Char(0, 14, uc_tmp / 10 + 0x30);
    LCD_Write_NewChar(uc_tmp % 10 + 0x30);
}
if (man==2)
{
    uc_tmp=20;
    LCD_Write_Char(0, 14, uc_tmp / 10 + 0x30);
    LCD_Write_NewChar(uc_tmp % 10 + 0x30);
}
if (man==3)
{
    uc_tmp=10;
    LCD_Write_Char(0, 14, uc_tmp / 10 + 0x30);
    LCD_Write_NewChar(uc_tmp % 10 + 0x30);
}
PORTC = (distance < 100) ? 0x01 : 0x02;
// 显示模式
LCD_Write_String(1, 0, (man == 0) ? "Auto" : "Manual");
LCD_Write_String(1, 8, "D:");

// 显示距离
i=15;
while (distance > 0) {
    unsigned char uc_d = distance % 10 + 0x30;
    LCD_Write_Char(1, i, uc_d);
    i--;
    distance /= 10;
}
while (i > 9) { // 高位没有数字时不显示
    LCD_Write_Char(1, i, ' ');
    i--;
}

```

```
    }  
    _delay_ms(1000);  
  }  
}
```

参考文献

- . ATmega8A 数据手册
- . DHT11 温湿度传感器技术文档
- . HC-SR04 超声波测距模块使用说明
- . LCD1602(I2C)驱动手册
- . 电子设计实践基础课程资料课件 PPT